

Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier, Julie Bonnaire, and Marie-France Fourcade

Cognitive Neuroimaging Unit
CNRS INSERM CEA

Abstract

We introduce **goxpyriment**, an open-source software framework for programming behavioral and cognitive experiments in the Go programming language. It addresses two recurrent practical difficulties of existing Python-based tools: the runtime-environment complexity that complicates deployment across laboratories, and the unpredictable pauses that automatic memory management can introduce during timing-critical sections. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), **goxpyriment** compiles an entire experiment into a single, self-contained executable with no runtime dependencies, so that distributing an experiment amounts to copying one file. Users can just click on the executable within their file browser and do not need to open a terminal and use the command line (but they can if they want/need to). The programming interface is inspired by Expyriment (Krause & Lindemann, 2014). It provides an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, ...) and audio capabilities (WAV playback and tone generation). For precise timing, response times are obtained by using operating-system event timestamps recorded at hardware-interrupt time rather than by continuous polling, and the garbage collector can be suspended for the duration of a trial. We characterize the resulting timing behaviour with the hardware-based protocol of the timing mega-study (Bridges et al., 2020), reporting visual, auditory, and trigger latencies across several hardware configurations. The framework ships with over forty example experiments that serve as reusable templates for common paradigms. **goxpyriment** is released under the Apache License 2.0 and is freely available at <https://github.com/chrplr/goxpyriment>.

Keywords: framework, library, stimulus presentation, timing, behavioral experiments, psychophysics, psychology

Introduction

Experiment-control software provides the foundation for empirical behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording,

the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through libraries such as PyEPL (Geller et al., 2007), VisionEgg (Straw, 2008), Expyriment (Krause & Lindemann, 2014) and PsychoPy (Peirce et al., 2019). Before that, MATLAB with the Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) was the de facto standard in many laboratories and is still in use in some of them. More recently, web-based frameworks such as jsPsych (de Leeuw, 2015; de Leeuw et al., 2023) and lab.js (Henninger et al., 2022) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB, extended with the Psychtoolbox, is convenient to generate visual or auditory stimuli and offers good timing on well-configured hardware. Yet Psychtoolbox scripts written on Windows most often do not work out of the box on Linux (and vice-versa). MATLAB can also be costly for institutions that cannot obtain academic licenses (as is the case for some non-degree-granting research institutions). Python tools provide a more open programming environment with a large scientific ecosystem. Python-based experiments can also achieve high-precision timing, for example by relying on the psychtoolbox Python module which provides pieces of the Psychtoolbox-3 (see <https://pypi.org/project/psychtoolbox/>). However, the use of Python introduces practical difficulties. For instance, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (pip, venv, conda, uv, ...) and can be a source of breakage across operating-system versions. For example, installing the `psychopy` module under Linux fails while compiling the `wxPython` package when no precompiled wheel is available. Second, Python's garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording during the critical trial loop. Web-based tools have an obvious advantage in terms of facility of deployment, but browser-mediated stimulus presentation is subject to event-loop latency that makes precise timing difficult (Anwyl-Irvine et al., 2021; Bridges et al., 2020).

The Go programming language (The Go Authors, 2026) offers a different set of tradeoffs. Go is a deliberately small language with a limited feature set and few alternative ways to express a given operation, which tends to make programs easy to read and to reason about (Donovan & Kernighan, 2016). It is statically typed, and its conventions encourage explicit error checking, so that many mistakes surface at compile time rather than at run time, which is a huge time saver for the programmer. Moreover, two properties of Go make it an attractive language to program and distribute experiments. First, Go (cross-)compiles to native machine code and can produce self-contained binaries that embed the stimuli and experiment lists and have thus no runtime dependencies. Second, its garbage collector is concurrent and can be suspended for the duration of a timing-critical section. These properties motivated our choice of Go as the basis for a behavioral experiment library that prioritizes deployment simplicity and timing control.

Christophe Pallier  <https://orcid.org/0000-0002-3324-666X> Correspondence should be addressed to C. Pallier, Cognitive Neuroimaging Unit, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org. I would like to thank Luca Bonatti and Raphael Bordas for alpha testing this software. The authors have no conflicts of interest to disclose.

The present article describes the design and capabilities of **goxpyriment**, details its timing architecture, and characterizes its temporal precision empirically. We highlight three contributions: (1) zero-dependency deployment via single compiled binaries that simplify distribution across laboratory hardware; (2) a timing architecture that obtains response times from operating-system event timestamps and that can suspend garbage collection during trials, whose behaviour we quantify using the protocol of the timing mega-study (Bridges et al., 2020); and (3) the existence of a gallery of more than forty examples of experiments.

Framework Design

Foundation: SDL3

Goxpyriment relies on the Simple DirectMedia Layer, a cross-platform development library designed to provide low-level access to audio, keyboard, mouse, joystick, and graphics hardware (SDL Developers, 2026). SDL has been used to develop thousands of commercial games as well as many emulators. Bindings for many programming languages exist, notably for Python with the **pygame** library (Pygame Developers, 2026). Because of its large user and developer bases, SDL is actively maintained and keeps up with modern hardware. **Goxpyriment** uses the latest version of this library, SDL3, through the **go-sdl3** bindings (Jung, 2026).

Single-Binary Deployment (Zero Dependencies)

Go compiles programs into a single statically linked executable with no external runtime dependencies. Crucially, the **go-sdl3** package embeds the pre-built SDL3 runtime libraries (SDL Developers, 2026) directly into the executable, so that no library needs to be installed separately on the target machine. In addition, the stimuli, experimental lists, fonts, and other assets can be embedded in the binary too. As a result, distributing an experiment requires copying just one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of installing experiments on multiple machines or sharing materials with collaborators.

Programming interface: a simple example

Listing listing 1 on the following page provides an example of a simple experiment implementing a parity decision task where participants have to decide if a number is even or odd.

- Line 12: An **Experiment** object that manages the window, renderer, and audio sub-systems must be created early. Calling it also parses the command-line to get the subject’s code and whether the experiment should run in fullscreen mode or not.
- Lines 16–36: This part of the code prepares the stimuli. Lines 31–36 are the most Go-specific: they create a map (a.k.a. associative array or dictionary in other programming languages) associating each number to a graphic texture. By pre-building these textures, we ensure that they will be displayed as quickly as possible in the

Listing 1: Example of a Parity Decision experiment

```

1 package main
2
3 import (
4     "strconv" // to convert int -> string
5
6     . "github.com/chrplr/goxpyriment/control"
7     "github.com/chrplr/goxpyriment/design"
8     "github.com/chrplr/goxpyriment/stimuli"
9 )
10
11 func main() {
12     exp := NewExperimentFromFlags("Parity Decision",
13         Black, White, 32)
14     defer exp.End()
15
16     Instructions := "When you'll see a number, your task to
17         decide, " +
18         "as quickly as possible, whether it is even or
19         odd.\n\n" +
20         "if it is even, press 'F'\n\nif it is odd, press
21         'J',"
22
23     Targets := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
24     NTrialsPerTarget := 2
25     trials := design.RepeatList(Targets, NTrialsPerTarget)
26     design.ShuffleList(trials)
27
28     EvenResponse := K_F
29     OddResponse := K_J
30     respKeys := []Keycode{EvenResponse, OddResponse}
31
32     cross := stimuli.NewFixCross(25, 2, DefaultTextColor)
33
34     // creates a map number -> image
35     Image := make(map[int]*stimuli.TextLine)
36     for _, num := range Targets {
37         Image[num] = stimuli.NewTextLine(strconv.Itoa(num),
38             0, 0, DefaultTextColor)
39     }
40
41     exp.AddDataVariableNames([]string{"number", "key", "rt",
42         "correct"})
43
44     exp.Run(func() error {
45         exp.ShowInstructions(Instructions)
46
47         for _, num := range trials {
48             exp.Blank(1500)
49             exp.ShowTimed(cross, 500)
50             key, rt, _ := exp.ShowAndGetRT(Image[num],
51                 respKeys, -1)
52
53             correct := (num%2 == 0) == (key ==
54                 EvenResponse)
55             exp.Data.Add(num, key, rt, correct)
56             if !correct {
57                 exp.Audio.PlayBuzzer()
58             }
59         }
60         return EndLoop
61     })
62
63     exp.ShowEndMessage("Experiment complete. Thank you!\n\n" +
64         "Press any key to exit.")
65 }

```

presentation loop (we could have called `stimuli.NewTextLine()` in the loop, saving a few lines at the cost of recomputing the texture at each iteration).

- Lines 41–56: This part of the code deals with the presentation loop. It is wrapped in a `Run` call that catches quitting events: if the participant presses the Escape key or closes the window, the library catches the resulting internal signal, safely flushes the data file to disk, and shuts down the SDL subsystems, ensuring that data is preserved even if a session is aborted prematurely. Line 47, the function `ShowAndGetRT()` presents the visual stimulus, waits for one of the given keys, then returns the reaction-time.

Timestamps, Garbage Collection Control, ...

Response timing in `goxpyriment` exploits SDL3 OS-level event timestamps. The `ShowAndGetRT` function used in the parity example is a wrapper for:

```
onset, _ := exp.ShowTS(stim)
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(keys,
                                                timeoutMs)
rt := int64(eventTS - onset) / 1_000_000
```

`ShowTS()` displays the stimulus `stim`, blocking until the next screen flip when VSYNC is enabled, and returning a time-stamp for that event (in ns). When VSYNC is disabled, `ShowTS()` returns immediately, supporting variable refresh rate (VRR) monitors.

`GetKeyEventTS()` scans the keyboard and returns the code of the key pressed and a timestamp (in ns) for it. This timestamp is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). Because SDL events carry their timestamp (information that pygame discards), the event remains in the queue with its original timestamp regardless of how much computation occurs between stimulus onset and response collection. By comparing the response timestamp with the `ShowTS()` timestamp, reaction times can be computed without interpreter overhead (hardware-level lags at the keyboard or mouse can, of course, still be present). An advantage of this approach is that there is no need to poll the HID device continuously to detect events: a complex sequence of stimuli can be programmed and every event remains available, with its original timestamp, to be processed later when needed.

To reduce the likelihood of latencies, `goxpyriment` provides additional functions:

Explicit Garbage Collection Control Go, like Python or JavaScript, has a memory garbage collector. This can introduce unpredictable pauses that corrupt timing. However, unlike Python or Javascript, Go allows the programmer to suspend the garbage collector. This can be used in `goxpyriment`'s timing-critical loops to prevent garbage-collection during a trial.

High-Precision Streams For paradigms with dense, high-speed stimulus sequences (e.g., retinotopic stimulation), `goxpyriment` provides specialized stream functions such as `PresentStreamOfImages`. These functions disable garbage collection, drain the event queue, and busy-wait at the sub-frame level to ensure that each stimulus onset is scheduled accurately. A `TimingLog` records the target and actual onsets for post-hoc verification, as well as the timing of keys pressed during the presentation of the stream.

Real-Time Scheduling Priority On Linux, an experiment built with `goxpyriment` requests the `SCHED_FIFO` real-time scheduling policy for itself at start-up (priority 50 by default), rather than relying on the user to remember a `chrt` prefix on the command line — which is in any case impossible when the program is launched by clicking its icon. The request is made with `sched_setscheduler()` on the main thread, to which the Go runtime has been pinned, so that the elevation applies to the thread that runs the trial loop. Under the default (`SCHED_OTHER`) policy the kernel may deschedule the process at any point, including between a stimulus onset and the trigger that marks it; this is the dominant source of the rare, large latencies that otherwise survive the measures above.

The request is not silently mandatory. It requires a system-level grant (an `rtprio` limit for the user, set in `/etc/security/limits.d/`), and if that grant is absent the program reports the reason and continues at normal priority rather than refusing to run. It can be disabled with the `-no-realtime` flag — useful when debugging, since a breakpoint hit on a real-time thread can leave the desktop unresponsive — and the priority can be changed with `-realtime-priority`. The policy actually obtained is recorded in the session's system report, so that a data set carries evidence of the conditions under which it was collected rather than of the conditions that were intended.

Triggers

To communicate with external recording equipment, such as fMRI, EEG, and MEG systems, `Goxpyriment` provides generic functions for reading from and writing to parallel and serial ports. Moreover, the current version implements TTL input/output interfaces for specific USB-serial devices: the DLP-IO-8 (see <https://github.com/chrplr/dlp-io8-g>) and an Arduino Mega 2560 utilizing a custom protocol (see <https://github.com/chrplr/neurospin-meg-ttl-box>), and for an Ethernet-based device: the LabJack T4 (<https://labjack.com/products/labjack-t4>). Adding support for new hardware is facilitated by the availability of the source code for the current devices.

A notable feature of the Go language is that it makes asynchronous function execution trivial by simply prepending the `go` keyword to a function call. For example, `go pp.Pulse(5, 200)` sends a 200 ms TTL pulse to line 5 of the parallel port in a 'fire-and-forget' manner. The function runs as a concurrent process and control returns immediately to the caller, allowing the program to proceed without delay—for example, to continue with stimulus presentation.

Miscellaneous additional features

This parity decision example only demonstrates a tiny part of the features provided by the framework.

For example, it is possible to build the experiment around a hierarchical structure of blocks and trials, over which the program can iterate, following a pattern inspired by Expyriment (Krause & Lindemann, 2014). Each trial is a collection of factors that can be randomized and logged. Data collection is handled by an `exp.Data` object, which writes to

A.

Retinotopy

Subject ID:
1

Age:
58

Gender (M / F / NB):
M

Handedness (R / L):
R

Screen width (cm):
53

Viewing distance (cm):
50

Refresh rate (Hz):
60

Run:
☒ RETBAR1
 ☐ RETBAR2
 ☐ RETCCW
 ☐ RETCW
 ☐ RETEXP
 ☐ RETCON

☒ Fullscreen mode

OK Cancel

B.

Number Comparison

Subject ID:
23

Stimulus group:
☒ digits
 ☐ regular
 ☐ irregular
 ☐ random

☒ Fullscreen mode

OK Cancel

Figure 1

Examples of `GetParticipantInfo` dialogs for Retinotopy (A.) and Number-Comparison (B.), two paradigms provided in the examples accompanying the library. These UIs makes it practical for the experimenter to choose among a small set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager. Note that which fields appear in the UI is configurable. It is also possible to pass the information as arguments on the command-line and skip these UIs.

a CSV file with metadata headers providing extensive information about the computer OS and hardware, timestamps, etc.

Another thing which is not transparent from the source code is the fact that `goxpyriment` provides a graphical `GetParticipantInfo` dialog that opens (optionally) before the main experiment window (see Fig. 1). This allows the experimenter to input subject IDs, select experimental conditions from discrete menus, and toggle settings like fullscreen mode. These values are automatically recorded in the output data file header.

Table 1

Results of a timing test performed on a Raspberry Pi 4 (Linux, arm64). Every half second, a 200ms tone was played, a 200ms white screen was displayed, and a TTL signal was sent. 1000 cycles. Measurements were performed with a Black Box Toolkit v3.

	Min	P5	P25	P50	P75	P95	Max	Range	P95-P05	Mean	SD
Durations ¹											
Opto1	220.0	220.2	220.2	220.5	220.5	220.8	220.8	0.8	0.5	220.4	0.17
Mic1	220.5	220.5	220.8	220.8	223.8	230.2	254.5	34.0	9.8	222.8	3.73
TTLin1	5.0	5.2	5.2	5.2	5.5	5.5	6.2	1.2	0.2	5.3	0.13
Inter-onsets intervals											
Opto1	499.5	499.5	499.8	499.8	500.0	500.2	500.2	0.8	0.8	499.9	0.19
Mic1	482.5	494.0	497.2	500.2	502.0	506.0	518.2	35.8	12.0	499.9	3.79
TTLin1	493.5	499.5	499.8	499.8	500.0	500.0	500.5	7.0	0.5	499.9	0.27
Paired-Event Onset Differences relative to TTLin1											
TTLin1→Opto1	37.8	38.2	38.2	38.5	38.5	38.8	39.0	1.2	0.5	38.5	0.17
TTLin1→Mic1	62.0	66.5	69.0	70.8	72.6	75.5	87.8	25.8	9.0	70.9	2.77

¹ Because smoothing was set on the ‘Opto’ and ‘Mic’ detectors, 20ms must be subtracted from these durations, as per the BBTK User Manual.

Timing Validation

To characterize temporal precision we adopt the hardware-measurement protocol of the timing mega-study (Bridges et al., 2020), which compared a range of lab-based and online experiment generators using a Black Box Toolkit (BBTK). Following that protocol, an external BBTK records physical events—light onsets from a photodiode placed on the screen, sound onsets from a microphone, and TTL edges on a digital input line—independently of the computer under test, so that the reported latencies do not depend on the timing of the machine being measured. The accompanying suite (`tests/Timing-Tests/`) drives these measurements and reports, for each event type, its duration and inter-onset interval (accuracy and precision), the audio–visual and trigger–visual onset differences (synchrony), the single-frame flash reliability, and the recovered response-time accuracy when the BBTK generates programmed responses at known delays. To evaluate the effect of memory management on timing, we additionally compare frame-interval distributions with the garbage collector enabled versus suspended.

Table 1 presents results of some of these tests running on a modest Raspberry PI 4 with 2Gb of RAM. The Raspberry displayed through HDMI on a Dell Monitor 1905FP, set at a resolution of 1280×1024 and a refresh rate of 60Hz. The only performance tweak applied was to set the timer pooling for `usb_serial` to the minimum possible value. A Black Box Toolkit version 3 (Plant et al., 2004), controlled by our software (Pallier, 2025), captured input events on the ‘Opto1’, ‘Mic1’ and ‘TTLin1’ lines. The first lines show the duration and inter-onset’s timings. The numbers are quite stable for the ‘Opto’ and ‘TTLin’ sensors (SD<0.5ms), and more variable for the ‘Mic1’ (sound detector). We believe that the variability is more likely to come from the loudspeaker than the actual payout by the computer. The last two lines of Table 1 reports the delays (lags) between the TTL signal and the video signal, and between the TTL and audio stimulus. These measures reveal a lag of 38ms, corresponding to a bit more than 2 frames at 60Hz, for the visual stimulus, and a lag of 70ms for the audio audio stimulus. As these lags are relatively stable (with

Table 2*Comparison of selected experiment frameworks.*

Property	goxpyriment	Expyriment	PsychoPy	PTB (MATLAB)	jsPsych
Language	Go	Python	Python	MATLAB	JavaScript
Deployment	Single binary	pip/conda	Standalone or pip	MATLAB license	CDN / npm
GC control	Explicit disable	No	No	N/A	No
VSYNC sync	Yes (SDL3)	Yes (SDL2)	Yes (Pyglet/PTB)	Yes	No
Hardware triggers	Yes	Yes	Yes	Yes	No
Response device API	Unified	Device-specific	Device-specific	Device-specific	N/A
Event timestamp	SDL3 hardware	pygame timer	iohub/PTB clock	GetSecs (μ s)	Browser clock
Stimulus set	Moderate	Moderate	Extensive	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No	No
Online deployment	No	No	Partial (PsychoJS)	No	Yes
Reference	This paper	Krause and Lindemann, 2014	Peirce et al., 2019	Brainard, 1997	de Leeuw et al., 2023

standard-deviations of 0.17ms and 2.77ms respectively), a user should compensate for them in an actual experiment. These lags will vary with the video and audio hardware and software (drivers & servers) and must be assessed for each configuration. A special folder in the goxpyriment github repository (**performance-reports**) is dedicated for users to share the results of timing tests.

Gallery of Examples

The repository provides over forty demos and examples of classic psychology paradigms, including the Stroop task, Attentional Blink (RSVP), Masked Priming, Retinotopic mapping for fMRI, and adaptive threshold estimation using the QUEST algorithm. These examples serve as both functional templates and a gallery of the framework's capabilities.

TODO: expand this section, insert a more structured description of Gallery of examples (and mention demos).

The examples and demos are available as pre-compiled binaries in the Releases tab of the github repository. Yet, compiling all of them locally can be achieved in three steps: 1) install Go on your machine; 2) clone or download goxpyriment repository 3) run the 'build.all' shell script.

Comparison with Existing Tools

Table 2 summarizes key properties of goxpyriment alongside five other experiment frameworks.

`goxpyriment` resembles Epyriment in its API design: a central experiment object, a `present()`-based stimulus API, a structured trial-loop model, and output to `.csv`-format files with metadata. The code for a simple experiment (Parity Decision) is provided in Listing 1. The primary technical differences are the use of Go rather than Python and the use of SDL3 rather than the older Pygame/SDL2 stack. From the user's point of view, `goxpyriment` adds *stream* objects which, as their name indicates, allow displaying sequences of stimuli with tightly controlled timing.

Table 2 necessarily simplifies; each framework has strengths that a compact matrix cannot convey. PsychoPy, in particular, offers a graphical Builder interface, a larger and extensively documented stimulus set, mature online deployment through PsychoJS, and a large user community. The Psychtoolbox remains a reference for low-level timing control on well-configured hardware. Relative to these established tools, `goxpyriment` is very young: it has no graphical builder, no online deployment, and a more limited stimulus library. Its distinctive contributions are therefore deployment simplicity and an explicit timing architecture rather than breadth of features. The entries in Table 2 describe design characteristics; measured timing comparisons on identical hardware are reported in Section .

Expandability

The availability of the source code makes it easy to expand `goxpyriment`, e.g., adding support for new hardware. Given a clear specification for an hardware interface, an human, or an AI coding assistant can quickly generate Go code for this hardware taking example from the existing packages.

Limitations and Future Directions

No Eye-tracker support. There is none for the moment. This is a target for future releases. The coding effort is expected to be small as some code has already been written to interact through TCP/IP to control some of the trigger devices.

No video recording. While there is currently some support for recording audio responses and measuring naming times, the framework does not currently allow to capture videos (unless one controls a videorecorder through the serial interface). As SDL3 supports recording video, this feature might be implemented in the future.

No way of sending results to a web server. Currently, results files are saved locally. In some situations, it could be useful to send the (anonymized) results to an on-line database, for example using a REST api.

Online deployment. Currently, `goxpyriment` does not support web-based deployment. As Go can compile to web assembly (wasm) and SDL3 supports EMSCRIPTEN (<https://emscripten.org/>), it is an achievable goal. We actually have a proof of concept, having successfully implemented several examples of the gallery to run in the browser. However, work remains to be done to support all `goxpyriment` functions in the browser. When this is achieved, the exact same source code will support web and local deployment.

Language barrier. Researchers who are unfamiliar with Go, that is, the vast majority, will need to invest a bit of time in getting acquainted with the language. To this end, many web resources exist, for example, <https://go.dev/tour/> and <https://go.byexample.com/>. As Go is a descendant of C, the syntax will be familiar to people who

know a language of the same family, e.g., Java or Javascript. Finally, the examples and the migration guide we provide at <https://chrplr.github.io/goxpyriment>, are also meant to ease the transition. AI-Coding tools can help a lot in porting existing matlab or python code, adapting existing experiments, or creating new ones.

Name of the project. The name “goxpyriment” is quite hard to pronounce, it is recommended to call the framework ‘gox-py’.

Declarations

Funding

No funding was received to assist with this project nor with the preparation of this manuscript.

Conflict of interest/Competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

Code availability

The source code of `goxpyriment` is available under the Apache License 2.0 at <https://github.com/chrplr/goxpyriment>. All example experiments described in this article are included in the repository. No empirical data are reported in this article.

Authors Contributions

Christophe Pallier: Conceptualization, Software implementation, Documentation and Paper Writing, with assistance from **Claude Sonnet 4.6** and **Gemini 2.5**. Consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude and Gemini are listed under Author Contributions rather than as a named author, because authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content. **Julie Bonnaire** and **Marie-France Fourcade** set up the timing measurement system and performed tests.

References

- Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, 52(1), 388–407. <https://doi.org/10.3758/s13428-019-01237-x>
- Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, 10(4), 433–436. <https://doi.org/10.1163/156856897X00357>
- Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, 8, e9414. <https://doi.org/10.7717/peerj.9414>
- de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, 47(1), 1–12. <https://doi.org/10.3758/s13428-014-0458-y>
- de Leeuw, J. R., Gilbert, R. A., & Luchterhandt, B. (2023). jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, 8(85), 5351. <https://doi.org/10.21105/joss.05351>
- Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.
- Geller, A. S., Schleifer, I. K., Sederberg, P. B., Jacobs, J., & Kahana, M. J. (2007). PyEPL: A cross-platform experiment-programming library. *Behavior Research Methods*, 39(4), 950–958. <https://doi.org/10.3758/BF03192990>
- Henninger, F., Shevchenko, Y., Meijer, R. R., Inzlicht, M., & Hilbig, B. E. (2022). Lab.js: A free, open, online study builder. *Behavior Research Methods*, 54(2), 556–573. <https://doi.org/10.3758/s13428-019-01283-5>
- Jung, B. (2026). *SDL3 Bindings for Go*. Retrieved April 1, 2026, from <https://github.com/Zyko0/go-sdl3>
- Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What's new in Psychtoolbox-3? *Perception*, 36(ECVP Abstract Supplement), 14. https://pure.mpg.de/rest/items/item_1790332/component/file_3136265/content
- Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, 46(2), 416–428. <https://doi.org/10.3758/s13428-013-0390-6>
- Nature Portfolio. (2023). Tools and technologies: Artificial intelligence.
- Pallier, C. (2025, February). *Bbtkv3*. Retrieved April 1, 2026, from <https://chrplr.github.io/bbtkv3/>
- Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- Plant, R., Hammond, N., & Turner, G. (2004). Self-validating presentation and response timing in cognitive paradigms: How and why? *Behavior research methods, instruments, & computers : a journal of the Psychonomic Society, Inc*, 36, 291–303. <https://doi.org/10.3758/BF03195575>
- Pygame Developers. (2026). *Pygame: Python game development development library*. Pygame Community. Retrieved July 26, 2026, from <https://www.pygame.org/>

- SDL Developers. (2026). *Simple DirectMedia Layer, sdl3*. Retrieved April 1, 2026, from <https://www.libsdl.org/>
- Springer Nature. (2023). Artificial intelligence (AI) policy for authors.
- Straw, A. D. (2008). Vision Egg: An open-source library for realtime visual stimulus generation. *Frontiers in Neuroinformatics*, 2. <https://doi.org/10.3389/neuro.11.004.2008>
- The Go Authors. (2026). *Go developers website*. Google. Retrieved July 26, 2026, from <https://go.dev>